classmate
Date
Page

DSA Assignment

2611006900 19

Theory Assignment 1.

1) Develop functions to implement BST covering creation, deletion and traversal.

→

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node *left, *right;

struct Node* createNode(int data){
    struct Node* node = malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}
struct Node* insert(struct Node* root, int data){
    if(root==NULL){
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}
struct Node* deleteNode(struct Node* root, int data){
    if (root==NULL)
        return root;
```

```
if (data < root->data)
    root->left = deleteNode(root->left, data);
else if (data > root->data)
    root->right = deleteNode(root->right, data);
else {
    if(root->left == NULL){
        struct Node* temp = root->right;
        free(root);
        return temp;
    }
    if (root->right == NULL){
        struct Node* Temp = root->left;
        free(root);
        return temp;
    }
    struct Node* temp = root->right;
    while(temp->left != NULL)
        temp = temp->left;

    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}
void inorder(struct Node* root){
    if(root != NULL){
        inorder(root->left);
        printf("%d", root->data);
        inorder(root->right);
    }
}
```

```
void preorder(struct Node* root){
    if(root!=NULL){
    printf("%d", root->data);
    preorder(root->left);
    preorder(root->right);
    }
}
void postorder(struct Node* root){
    if(root!=NULL){
    postorder(root->left);
    postorder(root->right);
```

a) Develop a program algorithm for DFS, BFS and SSSP & trace the given graph.

-> A) DFS Algorithm

DFS(G, v);

1. Mark v as visited.
2. Print v.
3. For each adjacent vertex u of v:
   If u is not visited:
   DFS(G, u).

B) BFS algorithm

BFS(G, start):

1. Create an empty queue.
2. Mark start as visited and enqueue it.
3. while the queue is not empty:
   a. Dequeue and print vertex v.
   b. For every adjacent vertex u:
      If u is not visited:
      Mark u as visited:
      Enqueue u.

C) SSSP algorithm.

DIJKSTRA (G, source):
1. Set distance[source] = 0.
2. Set all other distances to infinity.
3. Select the unvisited vertex u with minimum distance.
4. For every adjacent vertex v:
   If distance[u] + weight(u, v) < distance[v]:
   Update distance[v].
   Update predecessor[v].
5. Mark u as visited.
6. Repeat until all reachable vertices are visited.
7. Display shortest distance and paths.

D) DFS and BFS traversal

Assuming adjacent vertices are visited in ascending numerical order:

Starting vertex	DFS traversal	BFS traversal
1	1 → 2 → 3 → 4 → 5 → 6	1 → 2 → 3 → 4 → 5 → 6
5	5 → 2 → 1 → 3 → 4 → 6	5 → 2 → 4 → 6 → 1 → 3

E) SSSP from vertex 1

Destination	Shortest path	Distance
1	1	0
2	1 → 3 → 6 → 2	8
3	1 → 3	1
4	1 → 3 → 4	4
5	1 → 3 → 4 → 5	8
6	1 → 3 → 6	7

F) SSSP from vertex 5

Dest	S.P	Distance
1	5 → 4 → 3 → 1	8
2	5 → 2	1
3	5 → 4 → 3	7
4	5 → 4	4
5	5	0
6	5 → 6	2